

7. Technical Debt Analysis

Objective: Establish the prerequisites for an Agentic Harness & Marketplace initiative — code repository health, third-party tool usage, AI tool usage, database usage, and development environment readiness.

Date: July 15, 2026 | **Scope:** shende-shweta/FSDKC (repository root) — Laravel 12 / PHP 8.3 backend, React 19.2 / TypeScript / Vite frontend, Express dev-api, MariaDB 11 + MongoDB 7, Docker Compose, GitHub Actions CI

Executive Summary

EXECUTIVE SUMMARY

FSDKC (Klearcom monolith) is a small, well-documented dual-module platform (Discovery + Connect) with Docker-based local setup, one GitHub Actions workflow, and a mature AGENTS.md / docs/CODEBASE_AUDIT_ISSUES.md layer that already enumerates refactor targets for AI-assisted work. Repository hygiene basics are mostly in place (.gitignore, Node lock files, submodule .env.example files), but backend/composer.lock is absent, branch-governance files are missing, and quality gates are thin (no ESLint/Prettier/PHPStan/pre-commit enforcement; CI skips dev-api entirely).

The most severe agentic-readiness blockers are unauthenticated, rate-unlimited APIs with open CORS (backend/routes/api.php, dev-api/src/server.js) and unreliable test coverage (placeholder PHPUnit tests including random_int() flakiness; zero frontend tests). Legacy debt (extract() in PHP, one React class component with interval leak) is explicitly catalogued and scoped.

Overall verdict: the repo has a usable work queue and module boundaries for agent-driven refactors, but is not yet safe to treat as a self-verifying agentic harness until security baselines, deterministic CI gates, and PHP dependency locking are addressed.

Yes

TOP-LEVEL .GITIGNORE
PRESENT

1

CI/CD WORKFLOWS FOUND

9 / 11

THIRD-PARTY PACKAGES
DECLARED / WIRED

Partial

.ENV.EXAMPLE PRESENT

OVERALL CODEBASE RATING — TECHNICAL DEBT & AGENTIC READINESS

High Risk

D6 API security (no auth, open CORS, no rate limits) and unreliable CI test gates prevent trusting agent-authored merges today.

Readiness Benchmark Ratings

#	Dimension	GOOD	MODERATE	HIGH RISK	Measured	Rating
---	-----------	------	----------	-----------	----------	--------

D1	Code Repository Health	all checks pass	1–2 gaps	3+ gaps / no CI	<code>.gitignore</code> + 1 CI workflow present; missing <code>composer.lock</code> , CODEOWNERS, PR template	MODERATE
D2	Third-Party Tool Usage	mostly wired & current	some unused/unwired	many unused or unmaintained	9/11 infra-relevant packages wired; PHPStan declared but not configured or run; dual Laravel + Express stacks duplicate logic	MODERATE
D3	AI Tool / Agentic Readiness	ready	partial	not ready	Strong <code>AGENTS.md</code> tree + audit checklist; legacy <code>extract()</code> /class component debt; CI tests do not guard business logic	MODERATE
D4	Database Usage	sound	some gaps	no constraints / shared flat schema	MariaDB FKs + domain-scoped tables; manual <code>init.sql</code> only (no migration versioning); SQL seed not idempotent on re-apply	MODERATE
D5	Development Environment	reproducible	partial	manual / fragile	Docker Compose + README quick-start; no enforced lint/format; PHP deps not lock-pinned	MODERATE
D6	API Security Baseline (additional)	auth + CORS + rate limits on all routes	partial hardening	no auth / open CORS / no throttling	Entire API unauthenticated; <code>cors()</code> default and <code>allowed_origins => ['*']</code> ; no throttle middleware	HIGH RISK
D7	Test & Quality Gate Reliability (additional)	deterministic, broad CI coverage	partial gaps	flaky or placeholder tests	2 PHPUnit files (one uses <code>random_int()</code>); 0 frontend tests; PHPStan unused; <code>dev-api</code> not in CI	HIGH RISK

No additional readiness gaps beyond the standard dimensions were observed beyond D6 and D7 above.

7.1 Code Repository Health

Check	Finding	Consequence for FSDKC	Next step
<code>.gitignore</code> coverage	Inspected <code>.gitignore:1-8</code> — covers <code>backend/vendor/</code> , <code>node_modules/</code> , <code>frontend/dist/</code> , <code>backend/.env</code> , <code>.env</code> , <code>dev-api/.env</code> , log files	Reduces risk of committing vendor trees, build artifacts, and local secrets during agent runs	Extend with <code>backend/composer.lock</code> once generated (should be committed, not ignored)

CI/CD presence	Inspected <code>.github/workflows/ci.yml:1-55</code> — one workflow on <code>push / pull_request</code> running backend PHPUnit + frontend <code>npm run build</code>	Merges get basic PHP unit + TS compile checks, but dev-api is never built or tested in CI , so the default <code>npm run dev</code> path can regress silently	Add a <code>dev-api</code> job (at minimum <code>npm ci && node --check src/server.js</code>) to <code>.github/workflows/ci.yml</code>
Branch protection signals	Searched <code>.github/</code> — <code>workflows/ci.yml</code> only; 404 for <code>.github/CODEOWNERS</code> and <code>.github/pull_request_template.md</code>	No local evidence of required reviews or ownership routing; unreviewed agent PRs could land on <code>main</code> if branch rules are not set in GitHub UI	Add <code>.github/CODEOWNERS</code> and <code>.github/pull_request_template.md</code> with checklist for security/test gates
Lock files committed	Present: <code>package-lock.json</code> , <code>frontend/package-lock.json</code> , <code>dev-api/package-lock.json</code> . Missing: <code>backend/composer.lock</code> (HTTP 404 on <code>main</code>)	PHP installs are not reproducible across contributors, CI, and agent sandboxes — <code>composer install</code> may resolve different transitive versions day-to-day	Run <code>composer update --lock</code> in <code>backend/</code> and commit <code>backend/composer.lock</code> ; add CI step <code>composer validate --strict</code>

7.2 Third-Party Tools Usage

Package	Declared	Actually Wired?	Debt note
<code>laravel/framework</code> (^12.0)	Y (<code>backend/composer.json:6-8</code>)	Y	Routes/controllers in <code>backend/routes/api.php</code> ; standard Laravel bootstrap in <code>backend/bootstrap/app.php</code>
<code>mongodb/mongodb</code> (^2.0)	Y (<code>backend/composer.json:8</code>)	Y	Used throughout <code>backend/app/Services/MongoService.php</code> for transcripts, test events, diagnostics
<code>phpunit/phpunit</code> (^11.0)	Y (<code>backend/composer.json:11</code>)	Partial	CI runs <code>vendor/bin/phpunit</code> (<code>.github/workflows/ci.yml:38-44</code>) but tests are placeholders — see D7
<code>phpstan/phpstan</code> (^2.0)	Y (<code>backend/composer.json:12</code>)	N	No <code>backend/phpstan.neon</code> (404); not invoked in CI — dead devDependency
<code>express</code> (^4.21.2)	Y (<code>dev-api/package.json:14</code>)	Y	Full REST + SSE surface in <code>dev-api/src/server.js</code>

<code>mongodb</code> (Node ^6.12.0)	Y (<code>dev-api/package.json:15</code>)	Y	Connection, indexes, seed in <code>dev-api/src/mongo.js:1-48</code>
<code>mongodb-memory-server</code> (^10.1.4)	Y (<code>dev-api/package.json:16</code>)	Y	Fallback when <code>MONGODB_URI</code> unset (<code>dev-api/src/mongo.js:33-40</code>)
<code>cors</code> (^2.8.5)	Y (<code>dev-api/package.json:13</code>)	Y (misconfigured)	<code>app.use(cors())</code> with default allow-all (<code>dev-api/src/server.js</code> ~L17 per audit doc)
<code>@tanstack/react-query</code> (^5.62.0)	Y (<code>frontend/package.json:11</code>)	Y	Server-state fetching on Discovery/Connect pages
<code>zustand</code> (^5.0.2)	Y (<code>frontend/package.json:15</code>)	Y	UI selection state in <code>frontend/src/store/uiStore.ts:1-15</code>
<code>react-router-dom</code> (^7.1.0)	Y (<code>frontend/package.json:14</code>)	Y	Routing in <code>frontend/src/App.tsx:1-38</code>

Dual-runtime debt: KPI and tree-building logic is duplicated across Laravel controllers and `dev-api/src/server.js` (documented in `docs/CODEBASE_AUDIT_ISSUES.md:44-48`), increasing the cost of agent refactors that must touch both stacks.

7.3 AI Tool Usage & Agentic Readiness

Signal	Finding
AI agent documentation	Root <code>AGENTS.md:1-22</code> plus per-module guides at <code>backend/app/Modules/{Discovery,Connect}/AGENTS.md</code> and <code>frontend/src/modules/{Discovery,Connect}/AGENTS.md</code> — conventions explicitly ban <code>extract()</code> and class components
Enumerable work queue	<code>docs/CODEBASE_AUDIT_ISSUES.md:1-180</code> lists 14 categorized issues with file paths, line ranges, and remediation priority — ideal agent task backlog
AI tooling config	Minimal <code>.kiro/settings/mcp-bundles/.gitignore</code> only; no <code>.cursor/</code> , <code>CLAUDE.md</code> , or Copilot config in repo
Code generators / scaffolding	None found — manual module layout under <code>backend/app/Modules/</code> and <code>frontend/src/modules/</code>
Structural uniformity	Two parallel modules (Discovery, Connect) with mirrored API + page patterns; however legacy paths (<code>backend/app/Legacy/</code> , <code>LegacyReportController</code> , <code>LegacyMonitorPoller.jsx</code>) break conventions

Legacy pattern hotspots (use PDF `affected-files` expansion):

No files matched `backend/**/*.php` containing `extract\(\.`
No files matched `frontend/**/*.{jsx,tsx}` containing `extends Component` .

Assessment: The repo is **partially ready** for agentic refactors — module boundaries and a pre-written audit checklist provide enumerable, isolated work items — but legacy files, duplicated dual-stack logic, and weak CI mean agents cannot yet self-verify changes safely.

7.4 Database Usage

Check	Finding	Consequence for FSDKC	Next step
Schema design	Inspected <code>docker/mariadb/init.sql:1-65</code> — tables <code>discovery_jobs</code> , <code>discovery_nodes</code> , <code>connect_monitors</code> , <code>connect_check_results</code> with FKs on child tables; <code>users.email</code> UNIQUE	Referential integrity enforced at DB layer for core relationships; reduces silent orphan rows during agent schema edits	Add secondary indexes (e.g., <code>discovery_nodes.parent_id</code> , <code>connect_check_results.checked_at</code>) for tree/report queries
Migration hygiene	No <code>backend/database/migrations/</code> directory (404); README:1-28 states manual SQL init only	Schema changes require editing <code>init.sql</code> by hand — no versioned, reversible migration history; risky for agent-driven DDL	Introduce Laravel migrations mirroring <code>init.sql</code> , or adopt numbered SQL migration files with up/down scripts
Data ownership	MariaDB tables namespaced by domain (<code>discovery_*</code> , <code>connect_*</code>); MongoDB collections tagged by <code>module</code> field in <code>docker/mongodb/init.js</code> and <code>dev-api/src/mongo.js</code> indexes	Supports eventual service extraction along Discovery vs Connect boundaries	Document ownership in a <code>docs/DATA_OWNERSHIP.md</code> mapping tables/collections → module
Seed / sample data hygiene	MariaDB: <code>INSERT</code> statements in <code>init.sql:67-80</code> run once via Docker endpoint — not idempotent if re-applied. MongoDB: <code>seedMongoData()</code> in <code>dev-api/src/mongo.js</code> skips when data exists; supports <code>--force</code> via <code>dev-api/src/seed.js:4</code>	Re-running MariaDB init on an existing volume fails or duplicates; MongoDB seeding is safer for local/agent loops	Split MariaDB seed into a separate idempotent script (UPSERT or <code>INSERT IGNORE</code>)

7.5 Development Environment

Check	Finding	Consequence for FSDKC	Next step
<code>.env.example</code>	<code>backend/.env.example:1-14</code> and <code>dev-api/.env.example:1-2</code> present; no root <code>.env.example</code>	README quick-start (<code>README.md:40-48</code>) documents <code>dev-api/.env</code> copy — backend Docker path documented separately; onboarding requires reading two sub-paths	Add root <code>.env.example</code> pointing to both sub-env files, or a setup script
OS portability	<code>npm run dev</code> uses <code>concurrently</code> (<code>package.json:4-6</code>); Docker Compose in <code>docker-compose.yml:1-68</code> ; no OS-specific scripts	Linux/macOS/Windows (with Docker) can run the stack; suitable for mixed CI/contributor OS	None required beyond documenting Windows Docker Desktop prerequisites in README
Containerization	<code>docker-compose.yml</code> defines nginx, PHP app, MariaDB, MongoDB, frontend; PHP image at <code>docker/php/Dockerfile</code>	Full-stack reproducible environment for agents and CI-like parity	Pin image digests in compose for supply-chain reproducibility

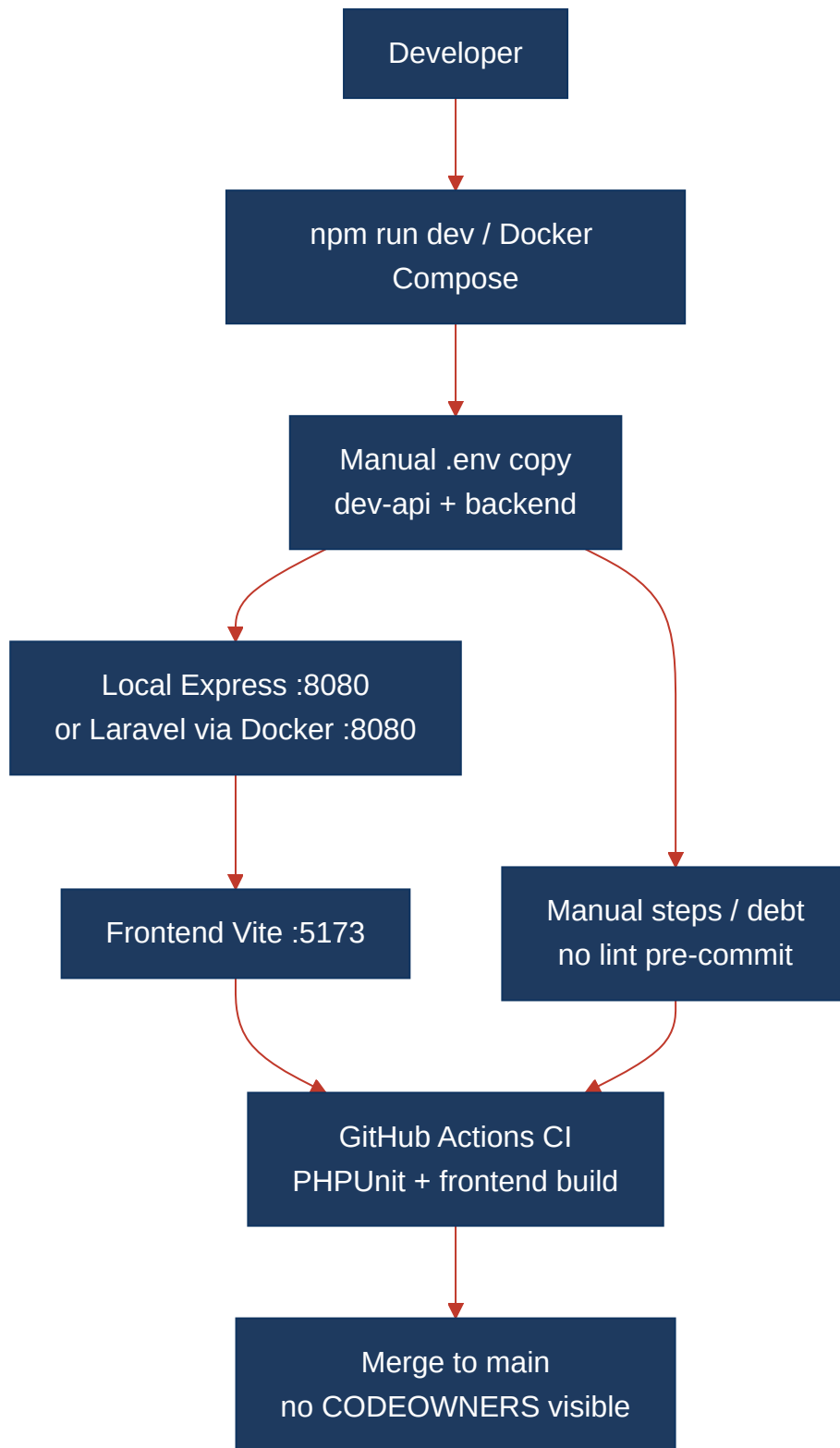
Code style enforcement	<code>404</code> for <code>.eslintrc*</code> , <code>.prettierrc</code> , <code>.pre-commit-config.yaml</code> , <code>husky/</code> ; PHPStan declared but not run; CI runs <code>tsc -noEmit</code> via <code>frontend/package.json:8</code> build only	Formatting and static analysis will drift immediately under agent churn; no pre-merge lint gate	Add ESLint + Prettier for frontend, wire PHPStan in CI, add pre-commit or CI lint jobs
------------------------	--	---	--

7.6 Prerequisites for Agentic Harness & Marketplace Readiness

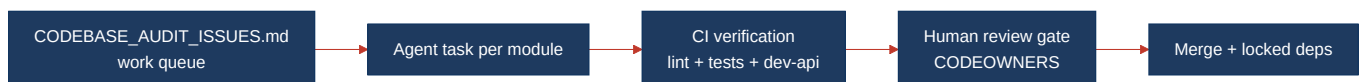
Prerequisite	Current State	Gap
Enumerable work queue (clear, listable units of refactor work)	<code>docs/CODEBASE_AUDIT_ISSUES.md</code> with 14 tracked items and file/line references	LOW — queue exists; needs linking to GitHub Issues for harness automation
Isolated, verifiable units of work	Discovery/Connect module split; services partially extracted (<code>MongoService</code> , <code>RealTimeTestService</code>)	MEDIUM — dual Laravel + Express duplication means agents must touch two runtimes for parity
CI gate to accept agent-authored output	<code>.github/workflows/ci.yml</code> runs PHPUnit + frontend build	HIGH — tests are placeholders/flaky; <code>dev-api</code> and lint/static analysis omitted
Repo hygiene for automation (clean checkout, no secrets)	<code>.gitignore</code> covers env files and vendor; lock files for Node	MEDIUM — missing <code>composer.lock</code> breaks PHP reproducibility
Marketplace packaging readiness	Docker Compose + README + module AGENTS docs	HIGH — no auth, open CORS, no rate limits block safe multi-tenant packaging

7.7 Diagrams

Current dev / delivery flow



Agentic harness readiness target



Improvement roadmap



7.8 Actions Required

Gap	Action	Rating	Priority
Missing <code>backend/composer.lock</code>	Run <code>composer update --lock</code> in <code>backend/</code> , commit lock file, add <code>composer validate --strict</code> to <code>.github/workflows/ci.yml</code>	MODERATE	HIGH
No branch governance files	Add <code>.github/CODEOWNERS</code> and <code>.github/pull_request_template.md</code> with security/test checklist	MODERATE	MEDIUM
PHPStan declared but unwired	Create <code>backend/phpstan.neon</code> and add <code>vendor/bin/phpstan analyse</code> step to CI	MODERATE	MEDIUM
<code>dev-api</code> absent from CI	Add CI job: <code>npm ci --prefix dev-api</code> and smoke test / syntax check on <code>dev-api/src/server.js</code>	MODERATE	HIGH
Unauthenticated API surface	Add auth middleware (token or session) to <code>backend/routes/api.php</code> and <code>dev-api/src/server.js</code> ; restrict CORS in <code>backend/config/cors.php</code> and <code>dev-api</code>	HIGH RISK	CRITICAL
No rate limiting	Apply Laravel <code>throttle</code> middleware in <code>backend/bootstrap/app.php</code> / route groups; add <code>express-rate-limit</code> to <code>dev-api</code>	HIGH RISK	CRITICAL
Flaky / placeholder tests	Replace <code>ReachabilityCalculationTest.php</code> random test with deterministic service tests; add frontend Vitest/RTL smoke tests; test <code>RealTimeTestService</code> and KPI math	HIGH RISK	CRITICAL
Unsafe <code>extract()</code> usage	Refactor <code>backend/app/Legacy/LegacyDataMapper.php</code> and <code>LegacyReportController.php</code> to explicit key access	MODERATE	CRITICAL
Legacy class component leak	Convert <code>frontend/src/components/LegacyMonitorPoller.jsx</code> to functional component with <code>useEffect</code> cleanup	MODERATE	HIGH
Manual SQL-only schema	Add versioned migrations (Laravel or numbered SQL) alongside <code>docker/mariadb/init.sql</code>	MODERATE	MEDIUM
No enforced code style	Add ESLint + Prettier configs under <code>frontend/</code> ; enforce via CI and optional pre-commit	MODERATE	MEDIUM
Dual-stack logic duplication	Consolidate KPI/tree/reachability logic into shared services; deprecate duplicate paths in <code>dev-api</code> or Laravel	MODERATE	HIGH

7.9 Expected Outcomes

- Reproducible PHP and Node installs via committed lock files, enabling identical agent sandbox and CI environments.
- CI gates (lint, static analysis, deterministic tests, dev-api smoke) that can trust agent-authored PRs before human review.

- Authenticated, rate-limited APIs suitable for marketplace packaging and multi-tenant agent harness execution.
- Clear migration path from manual `init.sql` to versioned schema changes, reducing modernization risk.
- Module-scoped refactor queue (`CODEBASE_AUDIT_ISSUES.md`) integrated with GitHub Issues for automated agent task dispatch.